

cl-nagoya/unsup-simcse-ja-large

like 1

Follow CL Research Group in N... 40



Feature Extraction



sentence-transformers



PyTorch



Transformers



wiki40b



Japanese

bert

sentence-similarity



License: cc-by-sa-4.0



Train

Use this model



Model card

Files

xet



Community 1

Downloads last month

242



Inference Providers

NEW

Feature Extraction

This model isn't deployed by any Inference Provider.



Ask for provider support

Dataset used to train cl-nagoya/unsup-simcse-ja-large

Collection including cl-nagoya/unsup-simcse-ja-large

Japanese SimCSE Collection

Tsukagoshi et al., Japanese SimCSE Tech... • 5 items • Updated Apr 18 • 2

unsup-simcse-ja-large

Usage (Sentence-Transformers)

Using this model becomes easy when you have [sentence-transformers](#) installed:

```
pip install -U fugashi[unidic-lite] sentence-transformers
```

Then you can use the model like this:

```
from sentence_transformers import SentenceTransformer
sentences = ["こんにちは、世界！", "文埋め込み最高！文埋め込み最高と叫びなさい", "極度乾燥しな

model = SentenceTransformer("cl-nagoya/unsup-simcse-ja-large")
embeddings = model.encode(sentences)
print(embeddings)
```

🔗 Usage (HuggingFace Transformers)

Without [sentence-transformers](#), you can use the model like this: First, you pass your input through the transformer model, then you have to apply the right pooling-operation on-top of the contextualized word embeddings.

```
from transformers import AutoTokenizer, AutoModel
import torch

def cls_pooling(model_output, attention_mask):
    return model_output[0][:,0]

# Sentences we want sentence embeddings for
sentences = ['This is an example sentence', 'Each sentence is converted']

# Load model from HuggingFace Hub
tokenizer = AutoTokenizer.from_pretrained("cl-nagoya/unsup-simcse-ja-large")
model = AutoModel.from_pretrained("cl-nagoya/unsup-simcse-ja-large")

# Tokenize sentences
encoded_input = tokenizer(sentences, padding=True, truncation=True, return_tensors='pt')

# Compute token embeddings
with torch.no_grad():
    model_output = model(**encoded_input)

# Perform pooling. In this case, cls pooling.
sentence_embeddings = cls_pooling(model_output, encoded_input['attention_mask'])
```

```
print("Sentence embeddings:")
print(sentence_embeddings)
```

🔗 Full Model Architecture

```
SentenceTransformer(
  (0): Transformer({'max_seq_length': 512, 'do_lower_case': False}) with Transform
  (1): Pooling({'word_embedding_dimension': 768, 'pooling_mode_cls_token': True, '
)
```

🔗 Model Summary

- Fine-tuning method: Unsupervised SimCSE
- Base model: [cl-tohoku/bert-large-japanese-v2](#)
- Training dataset: [Wiki40B](#)
- Pooling strategy: cls (with an extra MLP layer only during training)
- Hidden size: 1024
- Learning rate: 3e-5
- Batch size: 64
- Temperature: 0.05
- Max sequence length: 64
- Number of training examples: 2^{20}
- Validation interval (steps): 2^6
- Warmup ratio: 0.1
- Dtype: BFloat16

See the [GitHub repository](#) for a detailed experimental setup.

```
@misc{
  hayato-tsukagoshi-2023-simple-simcse-ja,
  author = {Hayato Tsukagoshi},
  title = {Japanese Simple-SimCSE},
  year = {2023},
  publisher = {GitHub},
  journal = {GitHub repository},
  howpublished = {\url{https://github.com/hppRC/simple-simcse-ja}}
}
```